

# Iteration 1 Worksheet

## Adding a feature

The **Item Details** feature was added to help users easily access all important information about an item before deciding to buy it. Now, when users tap on an item's card, they can see details like its name, condition, price, category, seller information and available payment options. We also included a section for related items, making it easier for users to explore similar products. The goal was to make the browsing process simple, transparent, and trustworthy.

During development, we gathered feedback through online meetings with our team members and made improvements based on their suggestions. The feature was built with a clear layout, pulling dynamic item details from the stub database. We tested it to ensure everything worked smoothly and displayed correctly on all devices. After final adjustments, the `feature#9_Item_Description` branch was merged into `feature#1` branch as both the features needed each other's data which completed the implementation. Now the feature is completed.

- **User Stories:**

- General Item Details: Display core item information (name, photo, price, category, and condition).
- More Item Information: Include seller details, and payment options.
  - General item details and more item information are linked to each other
- Related Items: Suggest similar items to enhance user engagement

- **Commit-Wise Discussion:**

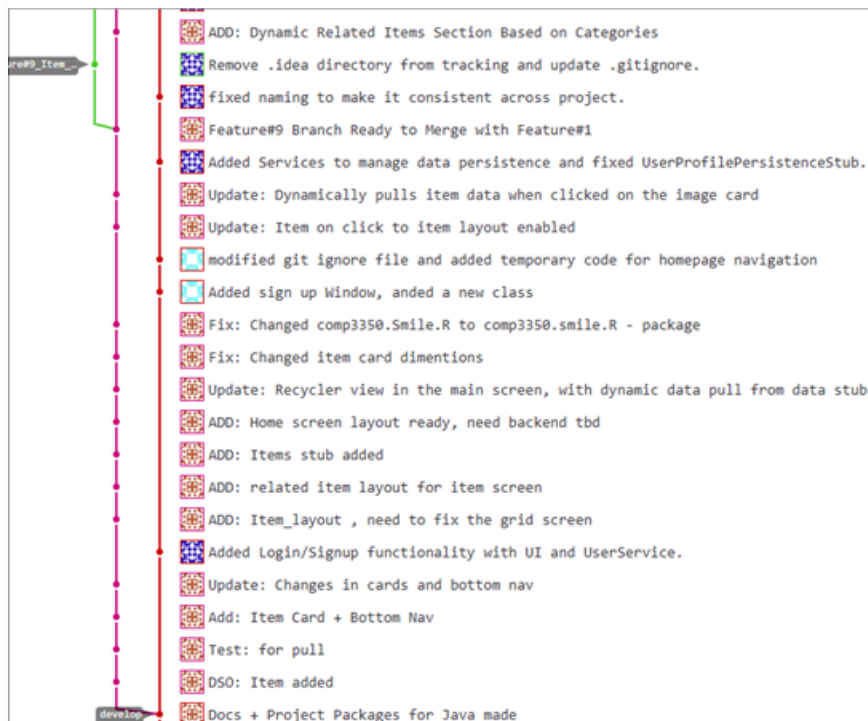
- [b81a4aae](#) - Item card (and Bottom Nav made)
- [66892dca](#) - Item Layout Screen added, the home screen
- [b1e3f061](#) - Related Item Layout added
- [efbca20f](#) - When merge was to be made from `feature#9_Item_Description` to `feature#1`
- [5cb28060](#) - Unit tests for the feature
- [871efb3a](#) - Final feature: Item Description Ready (Part of `feature#1` branch as we decided to continue with `feature#1` branch after the merge)

# Branching

We implemented the Git Flow strategy as a branching strategy, as agreed upon in our [Iteration 0 retrospective](#). This involved creating a develop branch and feature branches for each team member's assigned tasks

Throughout development, we followed a structured workflow:

1. The team members worked on the assigned development tasks within their respective branches.
2. Once each feature was completed and tested, it was merged into the develop branch.
3. On submission day, we merged the develop branch into the main branch for the final version.



# SOLID Violation - Group 7

[Violation](#) - AccessFlashcards.java

## [Issue](#)

Upon reviewing the repository, it was discovered that there is a clear violation in the file Quizario/app/src/main/java/comp3350/quizario/business/AccessFlashcards.java. Specifically, within the createFlashcard method, the following line of code is present:

```
subjectService = new AccessSubjects();
```

Dependency Inversion Principle (DIP):

This is a violation of the Dependency Inversion Principle because the AccessFlashcards class directly instantiates the concrete class AccessSubjects, rather than depending on an abstraction (such as an interface or abstract class for subject-related operations). This leads to tight coupling between the classes, making unit testing more challenging and reducing flexibility. Any changes to the concrete implementation of AccessSubjects would require modifications to the AccessFlashcards class, which goes against the principle of minimizing direct dependencies between modules.

Open/Closed Principle (OCP):

This also violates the Open/Closed Principle. By hardcoding the dependency on AccessSubjects, the AccessFlashcards class is not open to extension in a flexible way. If you needed to modify the behavior of subject validation (e.g., by using a different service), you would have to change the AccessFlashcards class itself, rather than simply providing a new implementation. This makes the class harder to extend without modifying its existing code.

# Agile Planning

For iteration #1, we initially planned to complete three features: **Profile Management** (Feature #1), **Item Description** (Feature #9), and **Look for Available Items** ([Feature#6](#)). Our plan was to start work on **Issue #1** and **Issue #9**, and if time permitted, we would work on **Issue #6**. We fully implemented **Feature #9** but chose to exclude the ratings function, which we may add later to a different, yet-to-be-determined feature. **Feature #1** was also fully implemented; however, we moved the completion of its test coverage to **Iteration #2**. Ultimately, we decided to push **Feature #6** to **Iteration #2** to ensure we could complete the first two features to the best of our ability. Additionally, we will be consolidating the **Chat** ([Feature#16](#)) and **Manage Chat** ([Feature#17](#)) features into a single **Frequently Asked Questions (FAQ)** feature. Lastly, we are introducing a new feature called **Cart**, which will allow users to track items they are considering for purchase.

## Retrospective 1

### 1. Start: What should we try in the next phase?

- Discuss our main topics for Tuesday/Thursday meetings.
- Clearly define specific roles for each member to prevent duplicate work and ensure everyone has a chance to contribute.

### 2. Stop: What should we avoid in the next phase?

- Poor communication for commits, to prevent using outdated versions of the project, avoid merge conflicts, and reduce wasted effort.
- Not assigning personal and group due dates.
- Not responding to messages promptly.

### 3. Continue: What should we keep doing in the next phase?

- Holding regular TR meetings on Discord.
- Maintaining a structured approach to work instead of rushing tasks.
- Using GitFlow as our branching strategy.
- Supporting group members who are struggling with a task.
- Speaking up, when a member is struggling with a task.

# Group Retrospective

## Retrospective per member

- What did you do well?
- What is your best code - link to it.

### Florence Emuze

- Helped implement the signup page and homepage navigation, wrote some documentation, and contributed to the login and sign-up page UI design.
- [Login Page code](#)

### Shiv Bhagat

- I was able to completely design UI for home page of the Smile app, also suggesting some UI ideas (Adding UM Logo) to the app's login page to provide support to the app as a part of UofM Community
- Dynamically gather random Items in the form of Cards for Home Screen and Related Items for Item Layout's Related Items Section.
  - [Home Screen Items](#) - Item Adapter Uses Base Item Adapter class
  - [Related Items](#) - Related Items Adapter Uses Base Item Adapter class

### Stephen Ugbah

- Familiarized myself with unit testing in android studio. I wrote tests to verify my teammates' codes. It got a little difficult after they made new changes and I had to try rewriting my tests to match their code. I need to do a better job of communicating with teammates to make sure we're aligned to reduce technical debt which for me would be not being ready for changes to the code and rendering my previous work not sufficient.
- My best code would be the [ItemServiceTest](#) file

## Shiv Patel

- I successfully implemented **dummy users** for testing purposes, which helped in verifying user interactions within the app. This made it easier to check how the app handles user data and provided a foundation for further development. I also worked on understanding GitLab and the version control workflow to manage my contributions effectively.
- [Commit Link - Adding Dummy Users](#)

## Jaskaranjot Khangura

- I implemented the logic for the Login activity and fully developed the Profile activity, including both the UI and services. Additionally, I added a 'current user' feature to ensure the user is recognized throughout the app.
- I'm particularly satisfied with the navigation code ([NavigationActivity.java](#)) I wrote, as well as the features I implemented in the Profile activity ([ProfileActivity.java](#)).